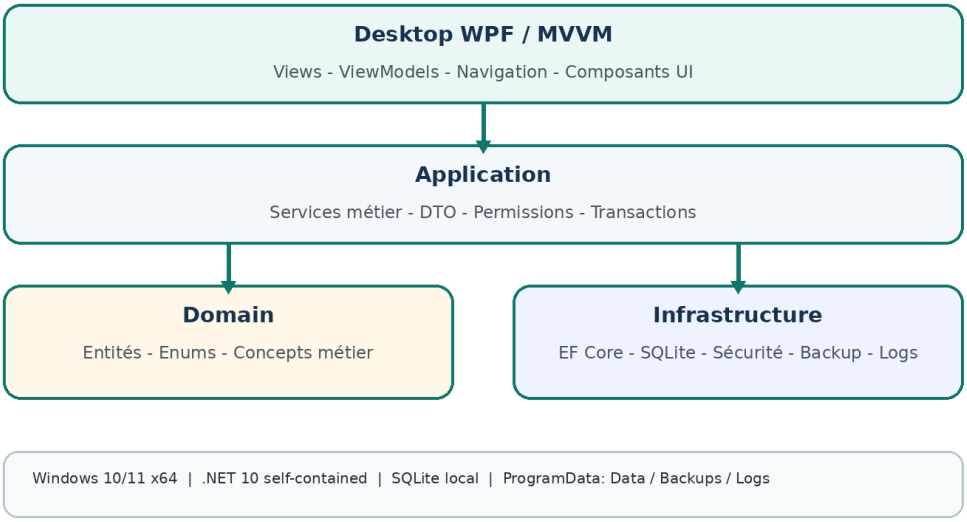
LOGICIEL DE GESTION FINANCIÈRE

Architecture technique

Version 1.0 - VALIDÉE

Architecture, données, sécurité, sauvegarde,
déploiement et règles de développement

Architecture technique V1.0



Statut du document
SOURCE DE VÉRITÉ VALIDÉE avant implémentation. Toute modification structurelle importante doit être décidée dans la conversation centrale du projet.

Date de validation : 02/09/2026

1. Objet et contraintes structurantes

Ce document fige l'architecture technique de la V1 avant le démarrage du développement. Il complète le cahier des charges, la modélisation fonctionnelle, le modèle de données et la conception UX/UI déjà validés.

> **Principe directeur**
> Fiabilité et intégrité des données d'abord, puis simplicité, sécurité, stabilité, performance, ergonomie et maintenabilité.

1.1 Contexte d'exécution

- Application desktop locale destinée à Windows 10, compatible avec une évolution vers Windows 11.
- Fonctionnement principal hors ligne, sans dépendance à un serveur distant.
- Un seul PC client en V1 ; aucune synchronisation multi-postes n'est requise.
- Saisie manuelle des ventes, achats, paiements, dépenses et informations de tiers.
- Un seul compte de caisse et un seul compte bancaire dans la V1.
- Devise applicative unique : dirham marocain (MAD / DH).

1.2 Contraintes de réalisation

- Délai opérationnel restant au moment de cette validation : environ 9 jours.
- Deux développeurs utilisent le même laptop via deux comptes Windows distincts.
- Chaque développeur possède son propre environnement Codex.
- Cette conversation ChatGPT reste le centre de décision, d'architecture et de validation.
- Les travaux parallèles doivent être isolés par tâches et branches ; le même fichier critique ne doit pas être modifié simultanément sans coordination.

1.3 Non-objectifs techniques

- Pas de microservices, Docker, serveur web, Redis, event bus ou architecture distribuée.
- Pas de cloud obligatoire, SaaS, synchronisation réseau ni auto-update Internet dans la V1.
- Pas de moteur comptable double entrée, CQRS complet ou event sourcing.
- Pas de serveur SQL à installer chez le client.

2. Stack technique retenue

Élément	Choix	Raison
Langage	C#	Écosystème mature pour logiciel métier Windows.
Runtime	.NET 10 LTS	Support long terme et publication self-contained.
Interface	WPF	Technologie mature, stable et adaptée aux applications de gestion.
Pattern UI	MVVM	Séparation claire entre vues, état et logique applicative.
Base	SQLite	Installation légère, transactions ACID, adaptée au mono-PC.
ORM	Entity Framework Core	Migrations, mapping et accès aux données structurés.
Tests	xUnit	Tests unitaires et d'intégration.
Logs	Serilog / logging .NET	Diagnostic local sans exposer les erreurs techniques.
Installateur	Inno Setup	Setup Windows simple, fiable et maîtrisable.

2.1 Choix WPF

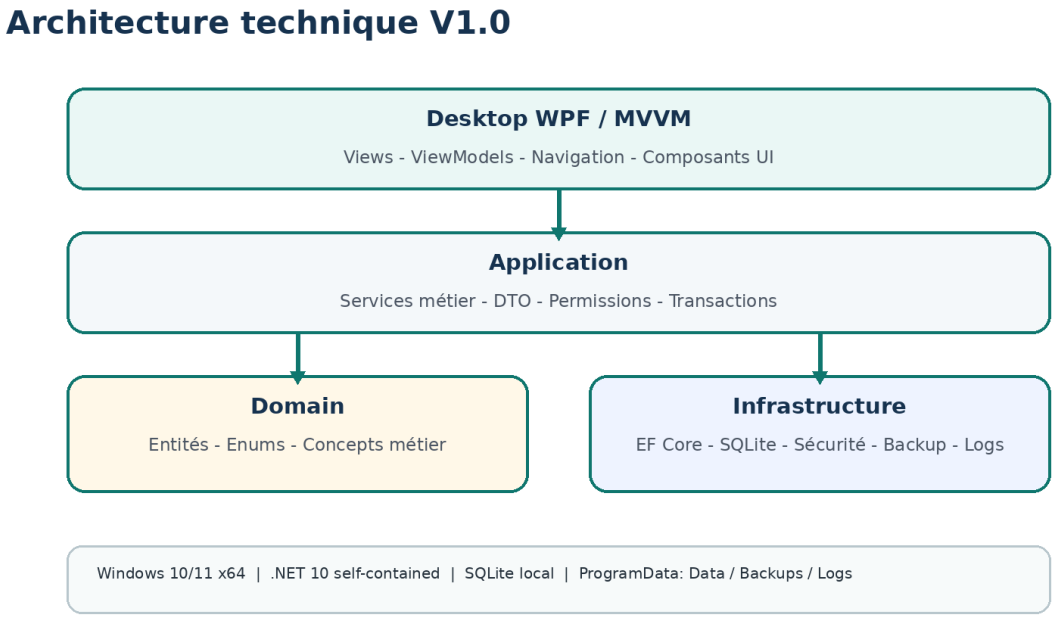
WPF est préféré à une interface web locale ou à une technologie Windows plus récente mais moins mature, car le projet doit être livré rapidement avec un risque technique minimal. Les formulaires, tableaux, navigation latérale et écrans de gestion sont des cas d'usage naturels pour WPF.

2.2 Choix SQLite

SQLite est retenu pour la V1 car le client utilise un seul PC et n'a besoin ni d'un serveur SQL ni d'accès concurrents depuis plusieurs machines. L'application devra toutefois conserver une séparation nette entre le domaine métier et l'infrastructure de persistance afin de rendre une migration future possible si le besoin multi-postes apparaît.

> **Décision**
> ARCH-STACK-01 : PASS - WPF + .NET 10 + MVVM + EF Core + SQLite.

3. Architecture applicative



3.1 Solution

La solution utilise quatre projets applicatifs et un projet de tests. L'objectif n'est pas de reproduire une Clean Architecture lourde, mais de conserver des frontières assez fortes pour empêcher la logique financière de se disperser dans l'interface.

Projet	Responsabilité
FinancialManagement.Domain	Entités, enums et concepts métier fondamentaux.
FinancialManagement.Application	Cas d'usage, règles métier, DTO, autorisations, transactions.
FinancialManagement.Infrastructure	EF Core, SQLite, sécurité, sauvegardes, logs, implémentations techniques.
FinancialManagement.Desktop	WPF, Views, ViewModels, navigation, composants et ressources.
FinancialManagement.Tests	Tests unitaires et tests d'intégration SQLite.

3.2 Règle de dépendance

- Domain ne dépend d'aucun projet applicatif.
- Application dépend de Domain.
- Infrastructure implémente les abstractions nécessaires et dépend de Domain/Application.
- Desktop dépend principalement d'Application ; Infrastructure intervient au démarrage pour la composition et l'injection de dépendances.
- Aucune logique financière structurante ne doit dépendre de WPF.

4. Organisation interne du code

4.1 Domain

- Entities : Client, Supplier, Sale, Purchase, CustomerPayment, SupplierPayment, Expense, User, TreasuryTransfer, etc.
- Enums : UserRole, PaymentMethod, TreasuryTransferDirection, BusinessModule.
- Exceptions ou concepts métier réellement partagés.

4.2 Application

Application constitue le cerveau métier. Les principaux domaines sont Sales, Purchases, Expenses, Customers, Suppliers, Payments, Treasury, Dashboard, Users et Authorization.

Service indicatif	Responsabilité principale
AuthenticationService	Authentification et chargement de session.
AuthorizationService	Rôle, permissions personnalisées et règles fortes.
SaleService / PurchaseService	Création, modification, paiement initial et cohérence.
CustomerPaymentService / SupplierPaymentService	Paiements, affectations et avances non affectées.
ExpenseService	Dépenses et impact trésorerie.
TreasuryService	Soldes calculés et transferts.
DashboardService	KPI et séries de données.
BackupService	Orchestration des sauvegardes et restaurations.

4.3 Desktop

Les ViewModels collectent les entrées utilisateur, appellent les services applicatifs et exposent l'état à la vue. Le code-behind reste réservé aux comportements purement visuels difficiles à exprimer proprement en MVVM.

> **Interdiction**
> Un ViewModel ou une View ne doit pas recalculer les créances, avances, soldes ou règles d'autorisation à la place des services métier.

5. Persistance et modèle physique

5.1 Principes

- Une seule base SQLite de production.
- Clés techniques INTEGER PRIMARY KEY.
- Références métier uniques séparées des identifiants techniques.
- Clés étrangères activées et suppressions configurées explicitement.
- Dates métier séparées des horodatages techniques UTC.
- Les soldes calculables ne sont pas dupliqués comme source de vérité.

5.2 Montants financiers

> **Règle de stockage**
> Les montants sont stockés en unités mineures entières. Exemple : 1 250,75 DH = 125075.
> Aucun montant financier n'est stocké en REAL/float.

5.3 Tables principales

Le schéma physique V1 comporte environ 17 tables. Les tables d'avance séparées sont volontairement évitées : une avance correspond à la portion non affectée d'un paiement.

Groupe	Tables
Sécurité	Users, UserPermissions
Clients / ventes	Clients, Sales, CustomerPayments, CustomerPaymentAllocations, InitialCustomerReceivables
Fournisseurs / achats	Suppliers, Purchases, SupplierPayments, SupplierPaymentAllocations, InitialSupplierPayables
Dépenses	ExpenseCategories, Expenses
Trésorerie	TreasuryOpeningBalance, TreasuryTransfers
Configuration	ApplicationSettings

6. Intégrité des données

6.1 Contraintes SQL

- Références métier uniques et nom d'utilisateur unique.
- Montants d'opérations strictement positifs.
- CHECK sur les valeurs d'enum essentielles.
- CHECK XOR sur les affectations : vente OU créance initiale, achat OU dette initiale.
- ON DELETE RESTRICT pour les tiers et utilisateurs déjà référencés.
- ON DELETE CASCADE uniquement lorsque la dépendance est réellement technique, notamment pour les affectations d'un paiement supprimé.

6.2 Contraintes de service métier

- Somme des affectations d'un paiement <= montant du paiement.
- Affectation client vers une obligation appartenant au même client.
- Affectation fournisseur vers une obligation appartenant au même fournisseur.
- Vente anonyme obligatoirement payée intégralement.
- Règles de modification Employé : autorisation + propriétaire + opération du jour.
- Utilisation d'une avance uniquement sur choix explicite de l'utilisateur.

6.3 Transactions atomiques

Toute action qui crée ou modifie plusieurs écritures financières doit être exécutée dans une transaction unique. Une erreur entraîne un rollback complet.

> **Exemple**
> Créer vente + créer paiement initial + créer affectation = une seule unité atomique. Aucune demi-opération ne doit rester dans la base.

7. Authentification et autorisations

7.1 Comptes

- Mot de passe stocké uniquement sous forme de hash robuste et salé.
- Compte actif/inactif ; un compte historique est désactivé plutôt que supprimé.
- Deux profils : RESPONSABLE et EMPLOYE.

7.2 Permissions personnalisées Employé

Module configurable	Voir	Créer	Modifier
Ventes	Oui/Non	Oui/Non	Oui/Non
Achats	Oui/Non	Oui/Non	Oui/Non
Dépenses	Oui/Non	Oui/Non	Oui/Non
Clients	Oui/Non	Oui/Non	Oui/Non
Fournisseurs	Oui/Non	Oui/Non	Oui/Non
Paiements clients	Oui/Non	Oui/Non	Oui/Non
Paiements fournisseurs	Oui/Non	Oui/Non	Oui/Non

7.3 Fonctions non délégables

- Dashboard financier global.
- Caisse & Banque.
- Administration et gestion des utilisateurs.
- Sauvegarde et restauration.
- Suppression définitive des opérations.

> **Double protection**
> Le masquage UI n'est jamais suffisant : chaque service métier vérifie également les droits avant d'exécuter une action sensible.

8. Trésorerie et calculs financiers

8.1 Sources de vérité

Aucune table générique de mouvement de trésorerie n'est utilisée comme seconde source de vérité. Les mouvements sont reconstruits à partir des opérations réelles.

Solde caisse	Solde banque
Solde initial caisse	Solde initial banque
+ paiements clients espèces	+ paiements clients virement
- paiements fournisseurs espèces	- paiements fournisseurs virement
- dépenses espèces	- dépenses virement
- transferts caisse vers banque	+ transferts caisse vers banque
+ transferts banque vers caisse	- transferts banque vers caisse

8.2 Données calculées

- Reste à payer d'une vente ou d'un achat.
- Créances et dettes courantes.
- Avances clients et fournisseurs.
- Statut payé / partiel / impayé.
- Soldes caisse et banque.
- Alertes d'échéance.
- KPI du dashboard et résultat Ventes - Achats - Dépenses.

> **Règle métier**
> Les soldes caisse et banque peuvent devenir négatifs. Aucun contrôle de disponibilité ne bloque une opération.

9. Configuration SQLite et robustesse

9.1 Paramètres de connexion

- Foreign Keys = ON.
- Journal Mode = WAL.
- Synchronous = FULL.
- Busy Timeout configuré.
- Migrations EF Core contrôlées.

9.2 Index

Les index seront concentrés sur les recherches réellement utilisées : dates métier, client, fournisseur, utilisateur, catégorie et échéance. Les références et usernames disposent déjà d'index uniques.

9.3 Propriété des migrations

> **Règle de collaboration**
> À un instant donné, un seul développeur est propriétaire des migrations et modifications structurelles de la base. Les deux Codex ne doivent jamais produire des migrations concurrentes indépendantes.

9.4 Première migration

La première migration, typiquement 001_InitialSchema, crée les tables, FK, CHECK, index, catégories initiales et paramètres de base. Toute modification ultérieure de schéma doit rester explicite et traçable.

10. Sauvegarde, restauration et récupération

10.1 Sauvegarde locale

- Sauvegarde automatique locale, au maximum une fois par jour.
- Conservation recommandée : 14 sauvegardes quotidiennes récentes.
- Sauvegarde manuelle depuis Administration.
- La sauvegarde utilise une procédure SQLite cohérente ; pas de simple copie sauvage de la base active.

10.2 Copie externe

Le Responsable peut copier une sauvegarde déjà valide vers une clé USB, un disque externe ou un autre dossier. La base de production active n'est pas directement copiée par cette fonction.

10.3 Restauration

1. Sélectionner une sauvegarde.
2. Vérifier que le fichier est lisible et compatible.
3. Créer automatiquement une sauvegarde pré-restauration de la base actuelle.
4. Restaurer la sauvegarde sélectionnée.
5. Vérifier l'ouverture de la base restaurée.

> **Règle de sécurité**
> Une restauration invalide ne doit jamais écraser la base courante.

11. Logs, erreurs et incidents

11.1 Logs techniques

- Fichiers locaux datés dans un répertoire Logs.
- Événements utiles : lancement, erreur inattendue, erreur DB, sauvegarde, restauration, migration et crash.
- Aucun mot de passe, hash ou secret ne doit être écrit dans les logs.

11.2 Erreurs utilisateur

Les exceptions techniques ne sont jamais affichées directement au client. L'utilisateur reçoit un message métier compréhensible tandis que les détails techniques restent dans le log.

11.3 Vérification au démarrage

- Base présente.
- Base accessible.
- Schéma et migrations compatibles.
- Connexion réussie.

> **Comportement attendu**
> Si la base ne peut pas être ouverte, l'application ne doit pas démarrer sur un tableau de bord vide pouvant faire croire à une perte de données.

12. Déploiement Windows

12.1 Publication

- Publication x64 self-contained.
- Le runtime .NET est inclus.
- Installateur Inno Setup.
- Raccourci Bureau et menu Démarrer.
- Désinstallation Windows standard.

12.2 Séparation programme / données

Emplacement	Contenu
C:\Program Files\Gestion Financiere	Exécutable, DLL et ressources de l'application.
C:\ProgramData\Gestion Financiere\Data	Base SQLite de production.
C:\ProgramData\Gestion Financiere\Backups	Sauvegardes.
C:\ProgramData\Gestion Financiere\Logs	Logs techniques.

Règle absolue

Une mise à jour ou désinstallation ne doit jamais supprimer silencieusement la base financière du client.

13. Premier démarrage et mises à jour

13.1 Configuration initiale

1. Nom de l'entreprise.
2. Création du premier compte Responsable.
3. Saisie du solde initial caisse.
4. Saisie du solde initial banque.
5. Validation et ouverture de l'application.

Les créances clients et dettes fournisseurs existantes sont saisies ensuite depuis les modules concernés afin d'éviter un assistant initial trop long.

13.2 Mise à jour

- Pas d'auto-update Internet en V1.
- Le client ou le support lance un nouvel installateur.
- L'installateur remplace le programme, jamais la base.
- Avant toute migration de schéma : sauvegarde pré-update automatique.
- Une migration échouée bloque le démarrage et laisse une sauvegarde de récupération.

13.3 Versionnement

Convention SemVer simple : 1.0.0 pour la première livraison, 1.0.1 pour un correctif, 1.1.0 pour une petite évolution et 2.0.0 pour une évolution majeure.

14. Stratégie de tests

14.1 Tests unitaires

- Vente anonyme non totalement payée refusée.
- Employé incapable de modifier l'opération d'un autre employé.
- Employé incapable de modifier hors jour autorisé.
- Surplus de paiement conservé comme montant non affecté.
- Permission personnalisée respectée.
- Calcul du reste, créance, dette et résultat.

14.2 Tests d'intégration SQLite

- Création vente + paiement + affectation atomique.
- Suppression paiement et recalcul du reste.
- FK et contraintes activées.
- Restauration d'une sauvegarde valide.
- Rollback en cas d'échec transactionnel.
- Redémarrage de l'application avec données persistantes.

14.3 Tests de livraison bloquants

Scénario	Gate
Installation propre	PASS obligatoire
Premier démarrage	PASS obligatoire
Création du Responsable	PASS obligatoire
Sauvegarde / restauration réelle	PASS obligatoire
Permissions Employé	PASS obligatoire
Mise à jour sans perte de DB	PASS obligatoire
Redémarrage Windows / application	PASS obligatoire
Réinstallation avec données existantes	PASS obligatoire

15. Organisation des 2 développeurs / Codex

15.1 Réalité de travail

- Même laptop physique.
- Deux comptes utilisateurs Windows distincts.
- Deux comptes Codex distincts.
- Le rythme matin / après-midi est indicatif, pas une contrainte.
- Deux conversations de travail pourront être ouvertes dans ce projet, une par développeur.
- Cette conversation centrale reste responsable des décisions et validations.

15.2 Mode de parallélisation

> **Stratégie**
> Travailler simultanément sur des tâches séparées et faiblement couplées. Ne pas faire travailler les deux développeurs sur la même zone critique au même moment.

À faire	À éviter
Branches ou clones séparés.	Deux Codex modifiant le même service en parallèle.
Responsabilités explicites par lot.	Migrations concurrentes.
Critères d'acceptation par tâche.	Refactors globaux non demandés.
Intégration contrôlée et tests.	Fusion directe dans main sans validation.
Retour des résultats importants à la conversation centrale.	Décisions d'architecture prises localement par un agent.

15.3 Règle de prompt Codex

Chaque mission Codex devra préciser au minimum : objectif, fichiers autorisés, fichiers interdits, dépendances, règles métier applicables, critères d'acceptation, tests requis et interdiction de refactorer hors périmètre.

16. Stratégie Git à préparer avant développement

La stratégie exacte sera figée dans la roadmap d'implémentation, mais l'architecture impose déjà les principes suivants.

- main reste stable et ne reçoit que du code validé.
- Chaque développeur travaille dans un espace isolé.
- Les modifications structurelles de base et migrations ont un propriétaire unique.
- Les travaux parallèles sont découpés pour minimiser les conflits.
- Une branche d'intégration temporaire peut être utilisée si elle apporte un gain réel.
- Les commits doivent rester petits, cohérents et liés à une tâche identifiable.

> **Préférence probable**
> Deux clones locaux séparés, un par compte Windows, reliés à une source Git commune, sont préférables à un unique dossier de travail partagé entre comptes Windows. La décision opérationnelle sera prise lors du setup de développement.

17. Décisions gelées avant implémentation

Sujet	Décision
Architecture	Monolithe modulaire en couches ; 4 projets + tests.
UI	WPF + MVVM.
Base	SQLite + EF Core ; un seul fichier de production.
Montants	INTEGER en unités mineures.
Trésorerie	Soldes calculés, négatifs autorisés.
Avances	Paiements non affectés ; pas de table d'avance.
Permissions	Responsable + Employé avec droits métier personnalisables.
Fonctions sensibles	Toujours Responsable uniquement.
Sauvegarde	Locale automatique + manuelle + copie USB.
Déploiement	Self-contained x64 + Inno Setup.
Collaboration	2 développeurs, tâches séparées, validation centrale.

17.1 Changements nécessitant validation centrale

- Changement de moteur de base.
- Changement d'architecture des projets.
- Nouvelle source de vérité financière.
- Modification des règles de permissions fortes.
- Ajout d'un nouveau module majeur.
- Modification de la stratégie de sauvegarde/restauration.
- Refactor transversal touchant plusieurs modules.

18. Gate final d’architecture

ARCHITECTURE-TECHNIQUE-V1
PASS - La V1 dispose désormais d’une architecture suffisamment précise pour lancer la planification d’implémentation sans laisser les développeurs ou Codex inventer les choix structurants.

18.1 Gates consolidés

Gate	Statut
ARCH-STACK-01	PASS
DATA-PHYSICAL-MODEL-01	PASS
ARCH-CODE-01	PASS
ARCH-SECURITY-RECOVERY-01	PASS
ARCH-DEPLOYMENT-01	PASS
ARCHITECTURE-TECHNIQUE-V1	PASS

18.2 Étape suivante

La prochaine phase est la roadmap d’implémentation des 9 jours : découpage des lots, dépendances, responsabilités Dev 1 / Dev 2, stratégie Git, séquence d’intégration, tests, critères de sortie et préparation des prompts Codex.

Objectif de la roadmap
Utiliser réellement le parallélisme des deux développeurs sans multiplier les conflits ni sacrifier les derniers jours de stabilisation.